

Programming Language Concepts Imperative Languages

Peter Sestoft
Niels Hallenberg
2026-07-24



Today

- A naïve imperative language
- C concepts
 - Pointers and pointer arithmetics, arrays
 - Lvalue and rvalue
 - Parameter passing by value and by reference
 - Expression statements
- Micro-C, a subset of C
 - Abstract syntax
 - Lexing and parsing
 - Interpretation

A naive-store imperative language

- **Naive** store model:
 - a variable name maps to an integer value
 - so store is just a runtime environment

```
sum = 0;  
for i = 0 to 100 do  
    sum = sum + i;
```

exec ex1 emptystore

i

100

sum

5050

ex1 as type stmt

```
let ex1 =  
    Block[Asgn("sum", CstI 0);  
          For("i", CstI 0, CstI 100,  
              Asgn("sum", Prim("+", Var "sum", Var "i")));  
          Print (Var "sum")]
```

A naive-store imperative language

```
i = 1;  
sum = 0;  
while sum < 10000 do begin  
    sum = sum + i;  
    i = 1 + i;  
end;
```

exec ex2 emptystore

i

142

sum

10011

ex2 as type stmt

```
let ex2 =  
  Block[Asgn("i", CstI 1);  
    Asgn("sum", CstI 0);  
    While (Prim("<", Var "sum", CstI 10000),  
      Block[Print (Var "sum");  
        Asgn("sum", Prim("+", Var "sum", Var "i"));  
        Asgn("i", Prim("+", CstI 1, Var "i"))]);  
    Print (Var "i");  
    Print (Var "sum")]
```

Naïve-store statement execution, 1

- Executing a statement gives a new store
- Assignment $x=e$ updates the store
- Expressions do not affect the store

```
let rec exec stmt (store : naivestore) : naivestore =  
  match stmt with  
  | Asgn(x, e) ->  
    setSto store (x, eval e store)  
  | If(e1, stmt1, stmt2) ->  
    if eval e1 store <> 0 then exec stmt1 store  
    else exec stmt2 store  
  | ...
```

Update store at
x with value of e

Naïve-store statement execution, 2

- A block $\{s_1; \dots; s_n\}$ executes s_1 then $s_2 \dots$
- Example:

```
exec (Block [s1; s2]) store
= loop [s1; s2] store
= exec s2 (exec s1 store)
```

```
let rec exec stmt (store : naivestore) : naivestore =
  match stmt with
  | Block stmts ->
    let rec loop ss sto =
      match ss with
      | [] -> sto
      | s1::sr -> loop sr (exec s1 sto)
    loop stmts store
  | ...
```

Naïve-store statement execution, 3

- `for` and `while` update the store sequentially

```
let rec exec stmt (store : naivestore) : naivestore =  
  match stmt with  
  | ...  
  | For(x, estart, estop, stmt) -> ...  
    let start = eval estart store  
    let stop  = eval estop  store  
    let rec loop i sto =  
      if i > stop then sto  
        else loop (i+1)  
          (exec stmt (setSto sto (x, i)))  
    loop start store  
  | While(e, stmt) ->  
    let rec loop sto =  
      if eval e sto = 0 then sto  
        else loop (exec stmt sto)  
    loop store
```



Environment and store, micro-C

- The naive model cannot describe *pointers* and *variable aliasing*
- A more realistic store model:
 - *Environment* maps a variable name to an address
 - *Store* maps address to value



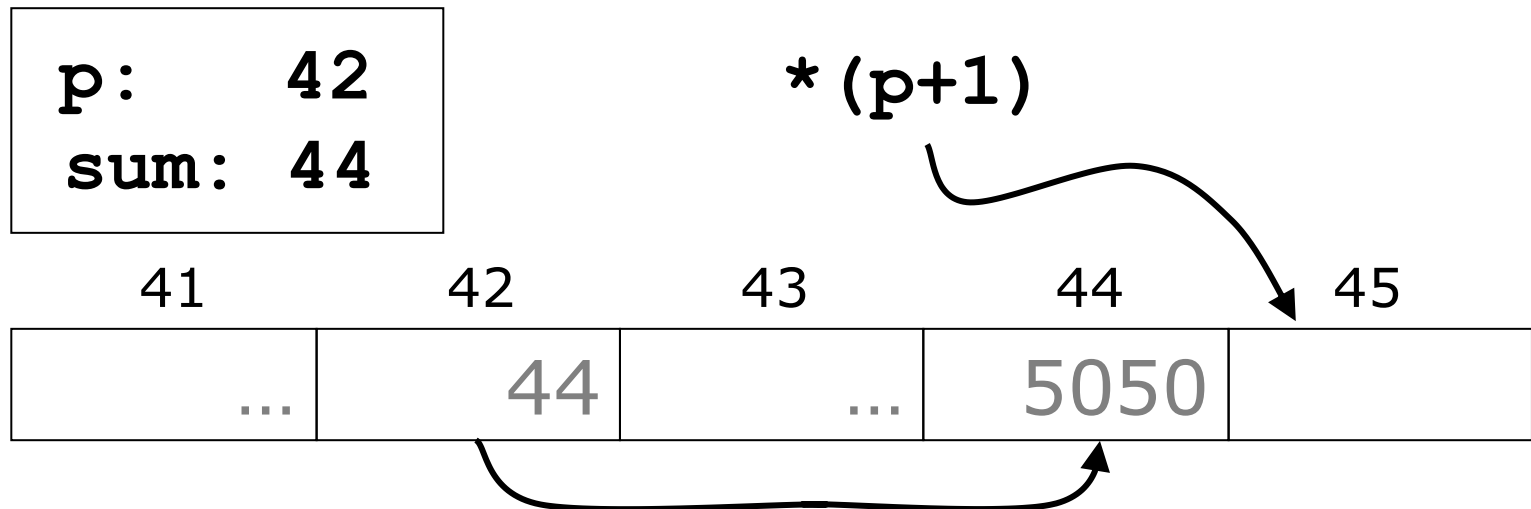
Today

- A naïve imperative language
- **C concepts**
 - **Pointers and pointer arithmetics, arrays**
 - **Lvalue and rvalue**
 - **Parameter passing by value and by reference**
 - **Expression statements**
- Micro-C, a subset of C
 - Abstract syntax
 - Lexing and parsing
 - Interpretation



The essence of C: Pointers

- Main innovations of C (1972) over Algol 60:
 - Structs, as in COBOL and Pascal
 - Pointers, pointer arithmetics, pointer types, array indexing as pointer indexing
 - Syntax: { } for blocks, as in C++, Java, C#



- Very different from Java and C#, which have no pointer arithmetics, but garbage collection

Desirable language features

	C	C++	F# / ML	Smtalk	Haskell	Java	C#
Garbage collection							
Exceptions							
Bounds checks							
Static types							
Generic types (para. polym.)							
Pattern matching							
Reflection							
Refl. on type parameters							
Anonymous functions (λ)							
Streams							
Lazy eval.							

C pointer basics

- A pointer **p** refers to a storage location
- The dereference expression ***p** means:
 - *the content of the location* (rvalue) as in
***p + 4**
 - *the storage location itself* (lvalue), as in
***p = x+4**
- The pointer that points to **x** is **&x**
- Pointer arithmetics:
 - *(p+1)** is the location just after ***p**
- If **p** equals **&a[0]**
then ***(p+i)** equals **p[i]** equals **a[i]**,
so an array is a pointer
- Strange fact: **a[2]** can be written **2[a]** too

C variable declarations

Declaration	Meaning
int n	n is an integer
int *p	p is a pointer to integer
int ia[3]	ia is an array of 3 integers
int *ipa[4]	ipa is an array of 4 pointers to integers
int (*iap)[3]	iap is a pointer to an array of 3 integers
int *(*ipap)[4]	ipap is a pointer to an array of 4 pointers to ints

Unix program `cdecl` or www.cdecl.org may help:

```
cdecl> explain int *(*ipap)[4]
declare ipap as pointer to array 4 of pointer to int
cdecl> declare n as array 7 of pointer to pointer to int
int **n[7]
```



Using pointers for return values

- Example MicroC/ex5.c, computing

```
void main(int n) {  
    int r;  
    r=n;  
    {  
        int r;  
        square(n, &r);  
        print r;  
    }  
    print r;  
}  
  
void square(int i, int *rp) {  
    *rp = i * i;  
}
```

for input

for return value: a
pointer to where to
put the result

```
> run (fromFile "ex5.c")  
[10];;
```

```
100 10
```

```
val it : Interp.store =  
map [(0, 10); <- n  
      (1, 10); <- r  
      (2, 100); <- r  
      (3, 10); <- i  
      (4, 2)] <- rp
```

Recursion and return values

- Computing factorial with MicroC/ex9.c

```
void main(int i) {  
    int r; 1  
    fac(i, &r);  
    print r;  
}  
  
void fac(int n, int *res) {  
    if (n == 0)  
        *res = 1;  
    else {  
        int tmp; 4  
        fac(n-1, &tmp);  
        *res = tmp * n;  
    }  
}
```

```
run (fromFile "ex9.c") [2];;
```

```
2 val it : Interp.store = map  
  [(0, 2); (1, 2); (2, 2); (3, 1);  
   (4, 1); (5, 1); (6, 4); (7, 1);  
   (8, 0); ...]
```

- n** is input parameter
- res** is output parameter:
a pointer to where to
put the result
- tmp** holds the result
of the recursive call
- &tmp** gets a pointer
to tmp



Possible evaluation of main(3)

main(3):

fac(3, 117):

&r is 117

fac(2, 118):

&tmp is 118

fac(1, 119):

&tmp is 119

fac(0, 120):

&tmp is 120

$*120 = 1$

$*119 = 1 * 1$

n is 1

$*118 = 1 * 2$

n is 2

$*117 = 2 * 3$

n is 3

print 6

r

tmp¹

tmp²

tmp³

...

117

118

119

120

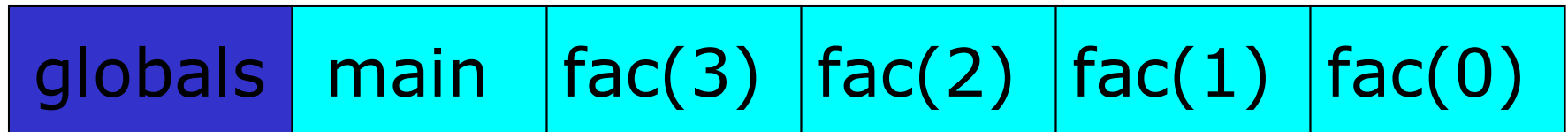
121

...	6	2	1	1	...
-----	---	---	---	---	-----



Storage model for micro-C

- The store is an indexable stack
 - Bottom: global variables at fixed addresses
 - Plus, a stack of activation records



- An *activation record* is an executing function
 - return address and other administrative data
 - parameters and local variables
 - temporary results



Lvalue and rvalue of an expression

- Rvalue is “normal” value, right-hand side of assignment: `17`, `true`
- Lvalue is “location”, left-hand side of assignment: `x`, `a[2]`
- In assignment `e1=e2`, expression `e1` must have lvalue
- Where else must an expression have lvalue in C#? In C?

	Has lvalue	Has rvalue
<code>x</code>	yes	yes
<code>a[2]</code>	yes	yes
<code>*p</code>	yes	yes
<code>x+2</code>	no	yes
<code>&x</code>	no	yes

Call-by-value and call-by-reference, C#

```
int a = 11;  
int b = 22;  
swapV(a, b);  
swapR(ref a, ref b);
```

addresses

a: 41
b: 42

by value

```
void swapV(int x, int y) {  
    int tmp = x; x = y; y = tmp;  
}
```

x: 43
y: 44
tmp: 45

41 (a)	42 (b)	43 (x)	44 (y)	45 (tmp)
11	22	22	11	11

Store (after evaluation)

by reference

```
void swapR(int *x, int *y) {  
    int tmp = *x; *x = *y; *y = tmp;  
}
```

x: 43
y: 44
tmp: 45

41 (a)	42 (b)	43 (x)	44 (y)	45 (tmp)
22	11	41	42	11

Store (after evaluation)



C variable declarations

Declaration	Meaning
int n	n is an integer
int *p	p is a pointer to integer
int ia[3]	ia is array of 3 integers
int *ipa[4]	ipa is array of 4 pointers to integers
int (*iap)[3]	iap is pointer to array of 3 integers
int *(*ipap)[4]	ipap is pointer to array of 4 pointers to ints

Unix program `cdecl` or www.cdecl.org may help:

```
cdecl> explain int *(*ipap)[4]
declare ipap as pointer to array 4 of pointer to int
cdecl> declare n as array 7 of pointer to pointer to int
int **n[7]
```

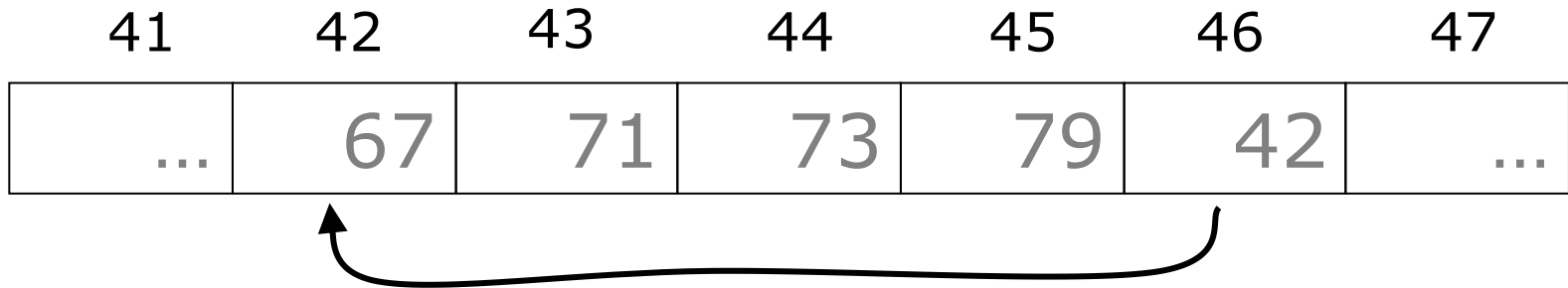
Today

- A naïve imperative language
- C concepts
 - Pointers and pointer arithmetics, arrays
 - Lvalue and rvalue
 - Parameter passing by value and by reference
 - Expression statements
- **Micro-C, a subset of C**
 - **Abstract syntax**
 - **Lexing and parsing**
 - **Interpretation**

Micro-C array layout

- An array `int arr[4]` consists of
 - its 4 int elements
 - a pointer to `arr[0]`

arr: 46



- This is the uniform array representation of B
- Real C treats array parameters and local arrays differently; complicates compiler
- Strachey's CPL -> Richards's BCPL -> B -> C

Micro-C syntactic concepts

- Types

<code>int</code>	<code>TypI</code>
<code>int *x</code>	<code>TypP (TypI)</code>
<code>int x[4]</code>	<code>TypA (TypI, Some 4)</code>

- Expressions

`(*p + 1) * 12`

- Statements

`if (x!=0) y = 1/x;`

- Declarations

- of global or local variables

`int x;`

- of global functions

`void swap(int *x, int *y) { ... }`

Micro-C abstract syntax

type typ =		
TypI	(* Type int	*)
TypC	(* Type char	*)
TypA of typ * int option	(* Array type	*)
TypP of typ	(* Pointer type	*)
and expr =		
Access of access	(* x or *p or a[e]	*)
Assign of access * expr	(* x=e or *p=e or a[e]=e	*)
Addr of access	(* &x or &*p or &a[e]	*)
CstI of int	(* Constant	*)
Prim1 of string * expr	(* Unary primitive operator	*)
Prim2 of string * expr * expr	(* Binary primitive operator	*)
Andalso of expr * expr	(* Sequential and	*)
Orelse of expr * expr	(* Sequential or	*)
Call of string * expr list	(* Function call f(...)	*)
and access =		
AccVar of string	(* Variable access	x *)
AccDeref of expr	(* Pointer dereferencing	*p *)
AccIndex of access * expr	(* Array indexing	a[e] *)
and stmt =		
If of expr * stmt * stmt	(* Conditional	*)
While of expr * stmt	(* While loop	*)
Expr of expr	(* Expression statement	e; *)
Return of expr option	(* Return from method	*)
Block of stmtordec list	(* Block: grouping and scope	*)
and stmtordec =		
Dec of typ * string	(* Local variable declaration	*)
Stmt of stmt	(* A statement	*)
and topdec =		
Fundec of typ option * string * (typ * string) list * stmt		
Vardec of typ * string		
and program =		
Prog of topdec list		

Types

rvalue

Expressions

lvalue

Statements

Declarations

Lexer specification for micro-C

- New: endline comments `// blah blah`
and delimited comments `if (x /* y? */)`

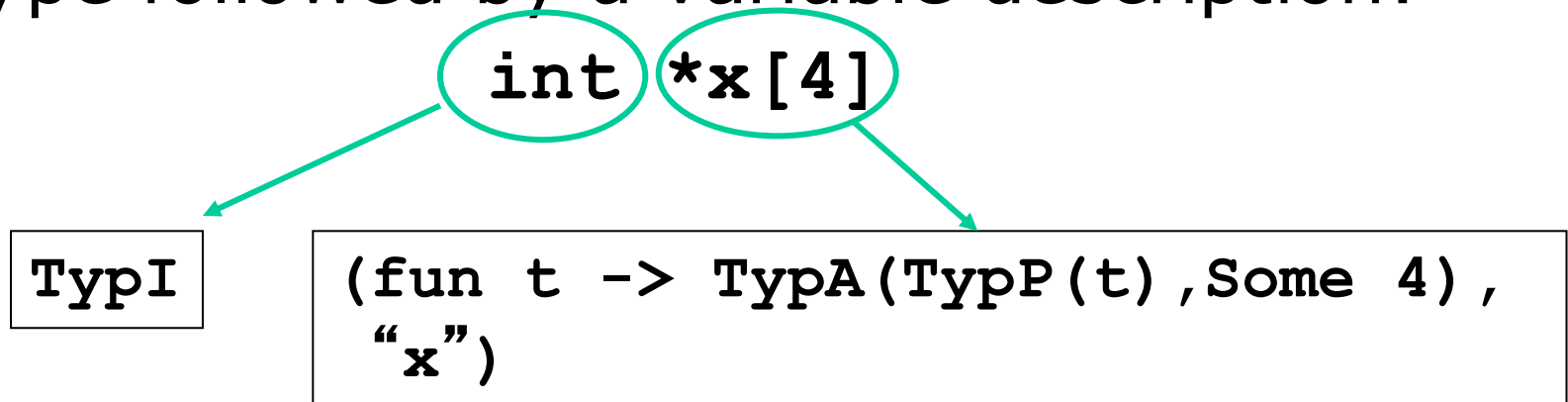
```
rule Token = parse
| ...
| "//"          { EndLineComment lexbuf; Token lexbuf }
| "/*"          { Comment lexbuf; Token lexbuf }
```

```
and EndLineComment = parse
| ['\n' '\r']   { () }
| (eof | '\026') { () }
| _             { EndLineComment lexbuf }
```

```
and Comment = parse
| "/*"          { Comment lexbuf; Comment lexbuf }
| "*/"          { () }
| ['\n' '\r']   { Comment lexbuf }
| (eof | '\026') { lexerError lexbuf "Unterminated" }
| _             { Comment lexbuf }
```

Parsing C variable declarations

- Hard, declarations are *mixfix*: `int *x[4]`
- Parser trick: Parse a variable declaration as a type followed by a variable description:



- Parse var description to get pair $(\mathbf{f}, \mathbf{x})$ of type function $\mathbf{f}$, and variable name $\mathbf{x}$
- Apply $\mathbf{f}$ to the declared type to get type of $\mathbf{x}$
`Vardec (TypA (TypP (TypI) , Some 4) , "x")`

Interpreting micro-C

- Interpreter data:
 - *locEnv*, *environment* mapping local variable names to store addresses
 - *gloEnv*, *environment* mapping global variable names to store addresses, and global function names to (parameter list, body statement)
 - *store*, mapping addresses to (integer) values
- Main interpreter functions:
 - `exec: stmt -> locEnv -> gloEnv -> store -> store`
 - `eval: expr -> locEnv -> gloEnv -> store -> int * store`
 - `access: access -> locEnv -> gloEnv -> store -> address * store`

Micro-C statement execution

- As for the naïve language, but two envs:

```
let rec exec stmt locEnv gloEnv store : store =  
  match stmt with  
  | If(e, stmt1, stmt2) ->  
    let (v, store1) = eval e locEnv gloEnv store  
    if v<>0 then exec stmt1 locEnv gloEnv store1  
    else exec stmt2 locEnv gloEnv store1  
  | While(e, body) ->  
    let rec loop store1 =  
      let (v, store2) = eval e locEnv gloEnv store1  
      if v<>0 then loop (exec body locEnv gloEnv store2)  
      else store2  
    in loop store1  
  | ...
```



Expression statements in C, C++, Java and C#

- The “assignment statement”

x = 2+y;

is really an expression

x = 2+y

followed by a semicolon

Value: none
Effect: change x

Value: 2+y
Effect: change x

- The semicolon means: ignore value

```
let rec exec stmt locEnv gloEnv store : store =  
  match stmt with  
  | ...  
  | Expr e ->  
    let (_, store1) = eval e locEnv gloEnv store  
    store1
```

Evaluate expression
then ignore its value



Micro-C expression evaluation, 1

- Evaluation of an expression
 - takes local and global env and a store
 - gives a resulting *rvalue* and a *new store*

```
and eval e locEnv gloEnv store : int * store =  
  match e with  
  | ...  
  | CstI i          -> (i, store)  
  | Prim2(ope, e1, e2) ->  
    let (i1, store1) = eval e1 locEnv gloEnv store  
    let (i2, store2) = eval e2 locEnv gloEnv store1  
    let res =  
      match ope with  
      | "*" -> i1 * i2  
      | "+" -> i1 + i2  
      | ...  
    (res, store2)
```



Micro-C expression evaluation, 2

- To evaluate access expression **x**, ***p**, **arr[i]**
 - find its lvalue, as an address **loc**
 - look up the rvalue in the store, as **store1[loc]**
- To evaluate **&e**
 - just evaluate **e** as lvalue
 - return the lvalue

rvalue

```
and eval e locEnv gloEnv store : int * store =  
  match e with  
  | Access acc ->  
    let (loc, store1) = access acc locEnv gloEnv store  
    (getSto store1 loc, store1)  
  | Addr acc -> access acc locEnv gloEnv store  
  | ...
```

Micro-C access evaluation, to *lvalue*

- A variable **x** is looked up in environment
- A dereferencing ***e** just evaluates **e** to an address
- An array indexing **arr[idx]**
 - evaluates **arr** to address **a**, then gets **aval=store[a]**
 - evaluates **idx** to rvalue index **i**
 - returns address (**aval+i**)

lvalue

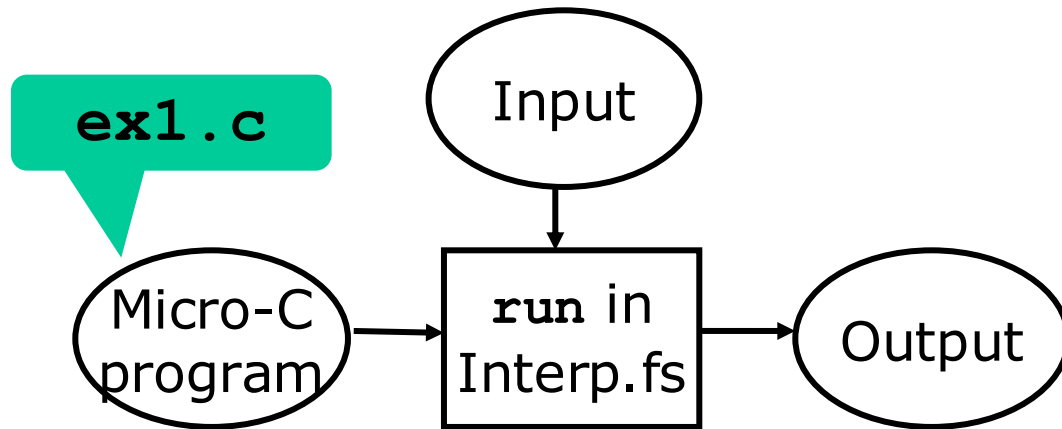
```
and access acc locEnv gloEnv store : int * store =  
  match acc with  
  | AccVar x          -> (lookup (fst locEnv) x, store)  
  | AccDeref e         -> eval e locEnv gloEnv store  
  | AccIndex(arr, idx) ->  
    let (a, store1) = access arr locEnv gloEnv store  
    let aval = getSto store1 a  
    let (i, store2) = eval idx locEnv gloEnv store1  
    (aval + i, store2)
```

Operators $\&x$ and $*p$ are inverses

- The address-of operator $\&e$
 - evaluates e to its lvalue
 - returns the lvalue (address) as if it were an rvalue
- The dereferencing operator $*e$
 - evaluates e to its rvalue
 - returns the rvalue as if it were an lvalue
- It follows
 - that $\&(*e)$ equals e
 - that $*(&e)$ equals e , provided e has lvalue

Micro-C, interpreter and compiler

- This lecture: Interpretation of micro-C



- Next lecture: Compilation of micro-C

